Operating Systems
Week 10 Lecture 10

Junaid Abdul Wahid

Previous

- Banker Algorithm
- Safety Algorithm

Today Agenda

- Deadlock Recovery
- Memory Management Background
- Contiguous Memory Allocation

Recovery from Deadlock: Process Termination

Abort all deadlocked threads
Abort one process at a time until the deadlock cycle is eliminated
In which order should we choose to abort?

1. Priority of the thread
2. How long has the thread computed, and how much longer to completion
3. Resources that the thread has used
4. Resources that the thread needs to complete
5. How many threads will need to be terminated
6. Is the thread interactive or batch?

Recovery from Deadlock: Resource Preemption

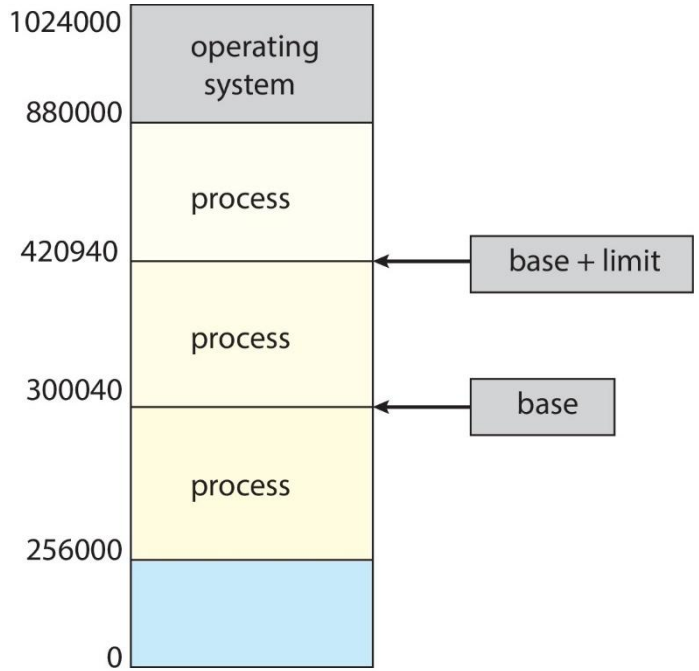
Selecting a victim – minimize cost

Rollback – return to some safe state, restart the thread for that state

Starvation – same thread may always be picked as victim, include number of rollback in cost factor

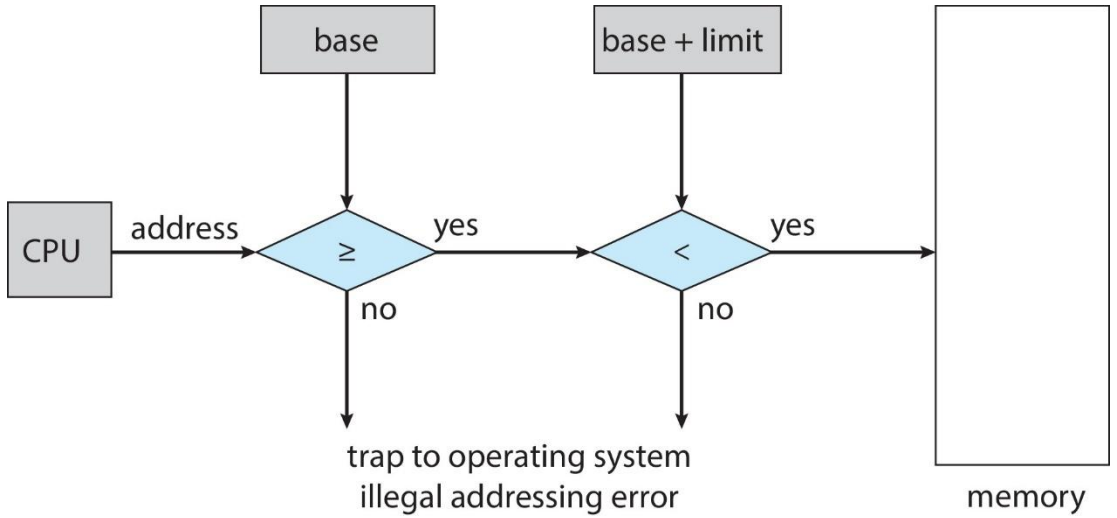
Protection

Need to ensure that a process can access only those addresses in its address space.
We can provide this protection by using a pair of **base** and **limit registers** define the logical address space of a process



Hardware Address Protection

CPU must check every memory access generated in user mode to be sure it is between base and limit for that user



the instructions to loading the base and limit registers are privileged

Address Binding

Programs on disk, ready to be brought into memory to execute form an **input queue**

- Without support, must be loaded into address 0000

Inconvenient to have first user process physical address always at 0000

Addresses represented in different ways at different stages of a program's life

- Source code addresses usually symbolic
- Compiled code addresses **bind** to relocatable addresses
  - i.e., "14 bytes from beginning of this module"
- Linker or loader will bind relocatable addresses to absolute addresses
  - i.e., 74014
- Each binding maps one address space to another

Binding of Instructions and Data to Memory

Address binding of instructions and data to memory addresses can happen at three different stages

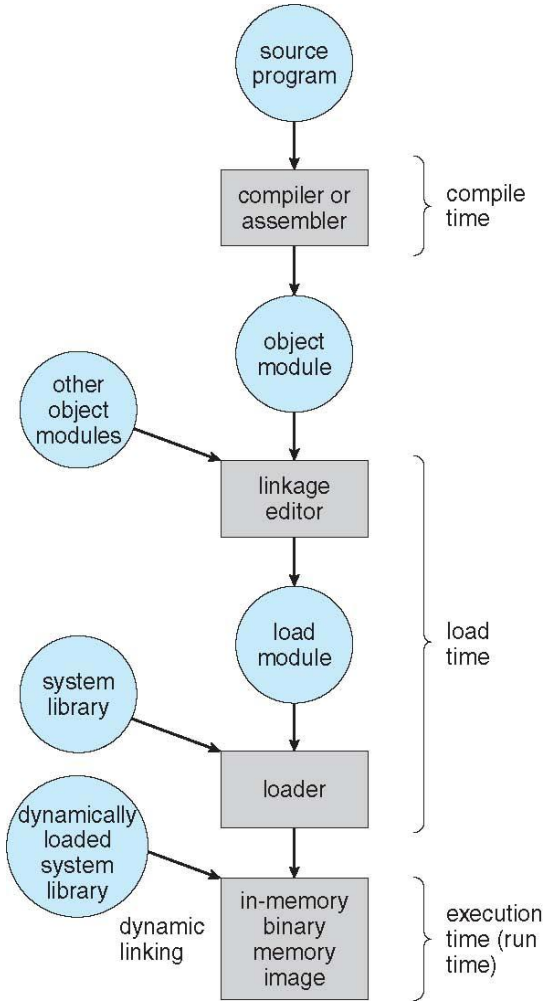
Compile time: If memory location known a priori, **absolute code** can be generated; must recompile code if starting location changes

Load time: Must generate **relocatable code** if memory location is not known at compile time

Execution time: Binding delayed until run time if the process can be moved during its execution from one memory segment to another

Need hardware support for address maps (e.g., base and limit registers)

Multistep Processing of a User Program



Logical vs. Physical Address Space

The concept of a logical address space that is bound to a separate **physical address space** is central to proper memory management

- **Logical address** – generated by the CPU; also referred to as **virtual address**
- **Physical address** – address seen by the memory unit

Logical address space is the set of all logical addresses generated by the **CPU**

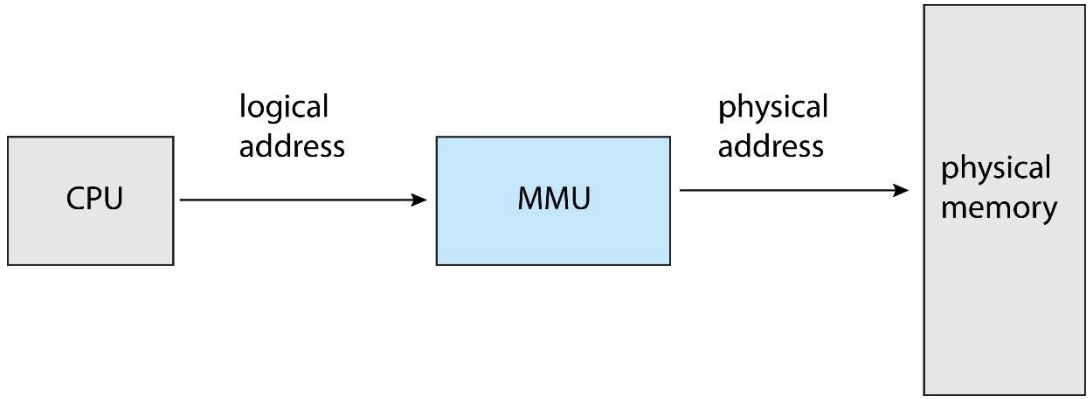
Physical address space is the set of all physical addresses generated by main memory (RAM)

Physical v Logical address

Aspect	Logical Address (Virtual Address)	Physical Address
Definition	Address generated by the **CPU**	Actual address in **main memory (RAM)**
Used by	Programs and processes	Memory Management Unit (MMU)
Visibility	Seen by the **user/programmer**	Hidden from the user, handled by OS
Mapped via	Page tables / segmentation units	No mapping needed (direct hardware)
Example	0x00403021 (virtual)	0x0A403021 (physical)

Memory-Management Unit (MMU)

Hardware device that at run time maps virtual to physical address



Imagine a hotel:

- **Logical address** = Room number on your **booking receipt**
- **Physical address** = The actual room you stay in
- **MMU (OS)** = The receptionist who maps your receipt (logical) to the actual room (physical)

Memory-Management Unit (Cont.)

Supports these memory allocation schemes

Translates logical → physical addresses.

The user program deals with *logical* addresses; it never sees the *real* physical addresses

Think of it as a **translator** between what a program sees and where the data actually lives in physical memory (RAM).

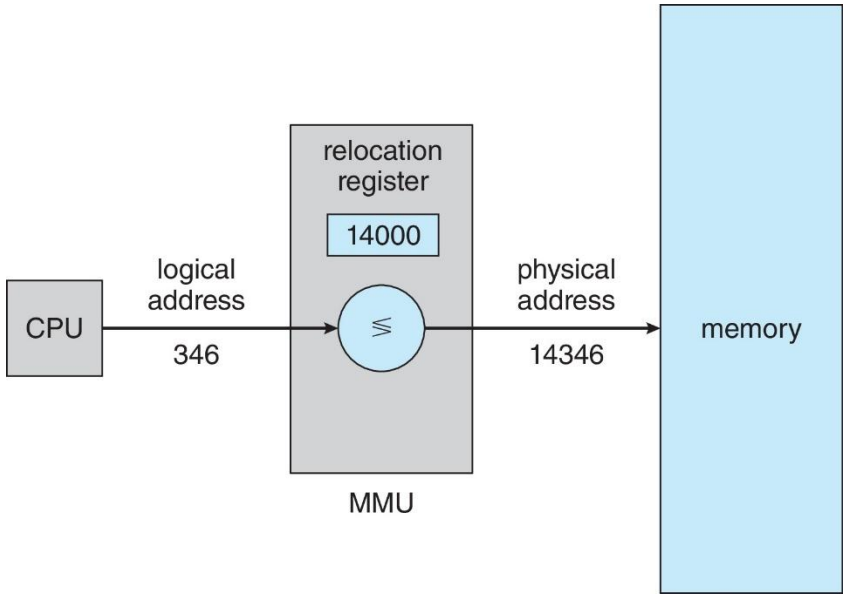
Prevents processes from accessing others' memory

Memory-Management Unit (Cont.)

Consider simple scheme. which is a generalization of the base-register scheme.

- The base register now called **relocation register**

The value in the relocation register is added to every address generated by a user process at the time it is sent to memory



Dynamic Loading

- The entire program does need to be in memory to execute
- Routine is not loaded until it is called
- Better memory-space utilization; unused routine is never loaded
- All routines kept on disk in relocatable load format
- Useful when large amounts of code are needed to handle infrequently occurring cases
- No special support from the operating system is required
  - Implemented through program design
  - OS can help by providing libraries to implement dynamic loading

Dynamic Linking

Static linking – system libraries and program code combined by the loader into the binary program image

Dynamic linking –linking postponed until execution time

Small piece of code, **stub**, used to locate the appropriate memory-resident library routine

Stub replaces itself with the address of the routine, and executes the routine

Operating system checks if routine is in processes' memory address

- If not in address space, add to address space

Dynamic linking is particularly useful for libraries

System also known as **shared libraries**

Consider applicability to patching system libraries

- Versioning may be needed

Contiguous Allocation

Main memory must support both OS and user processes
Limited resource, must allocate efficiently
Contiguous allocation is one early method
Main memory usually into two **partitions**:

- Resident operating system, usually held in low memory with interrupt vector
- User processes then held in high memory
- Each process contained in single contiguous section of memory

Contiguous Allocation (Cont.)

Relocation registers used to protect user processes from each other, and from changing operating-system code and data

- Base register contains value of smallest physical address
- Limit register contains range of logical addresses – each logical address must be less than the limit register
- MMU maps logical address *dynamically*
- Can then allow actions such as kernel code being **transient** and kernel changing size